

Move VM is a **typed stack machine** that runs verified bytecode. It does not discover “accounts to lock” up front. On Aptos, every global storage access is intercepted by a **data view**; Block-STM sits under that view and records the read/write set while the interpreter runs. That is why Move can stay sequential-looking for developers and still parallelize.

What the VM actually is

Three layers:

- 1. **Bytecode + verifier** — modules/scripts in a binary format; loaded, verified, cached.
- 2. **Interpreter** — stack + locals + call frames; gas-metered opcodes.
- 3. **Adapter** — chain-specific: Aptos account/resource/object storage, natives, prologue/epilogue, Block-STM view.

The core language is about **scarcity** (abilities `copy` , `drop` , `store` , `key`) and **module-as-capability** (only the defining module can unpack a struct unless it exposes constructors). Assets are resources, not ERC-20 balances in a mapping.

Programs the VM will run

Kind	Lives on chain?	Role
Module	Yes, under an address	Types + functions; the “OS libraries” and apps
Script	No (ephemeral payload)	One-shot <code>main</code> ; calls into modules
Entry function	Yes, inside a module	Today’s usual user tx: <code>addr::mod::fn(args)</code>

Loading: deserialize → verify (type safety, ability rules, no dangling refs, stack balance) → link against already-loaded modules → cache. Aptos now **lazy-loads** on first use (AIP-127) instead of metering the whole dependency tree at publish/execute. Code is immutable for the life of that VM instance; upgrades are new published bytecode, not JIT mutation.

Interpreter model

The runtime is a stack machine. Rough shape of one transaction:

text

prologue (framework)	– auth, sequence/nonce, fee check
execute payload	– script or entry function
epilogue (framework)	– charge gas, bump sequence, emit events

Inside execute :

- Push/pop values (u8 ... u256 , address , bool , signer , structs, vectors, references).
- Locals on a frame; Call / CallGeneric push a new frame.
- Control flow: BrTrue , BrFalse , Branch , Abort .
- Global storage opcodes (Aptos/Diem-style):

text

MoveTo / MoveToGeneric	publish a resource under an address
MoveFrom	take resource out of storage
BorrowGlobal / Mut	& / &mut resource
Exists	check resource presence

References are **bytecode-checked**: you cannot forge a &mut , hold a reference across an Abort in an unsafe way, or alias a resource unsoundly. That is why Block-STM can let the VM run on a *speculative* snapshot: if the snapshot is inconsistent, the VM either aborts cleanly or the adapter reports a read error; the engine does not need STM “opacity” inside every opcode.

Natives (crypto, hashing, table ops, event emit) are Rust functions registered by the adapter. They must be deterministic.

Global storage (Aptos)

State is a map from **access paths** to blobs:

text

Module	: (address, module_name) → bytecode
Resource	: (address, struct_tag) → BCS value
Table	: (handle, key) → value // large maps
Object	: ObjectAddr → group of resources // AIP-10

A user account is an address that holds:

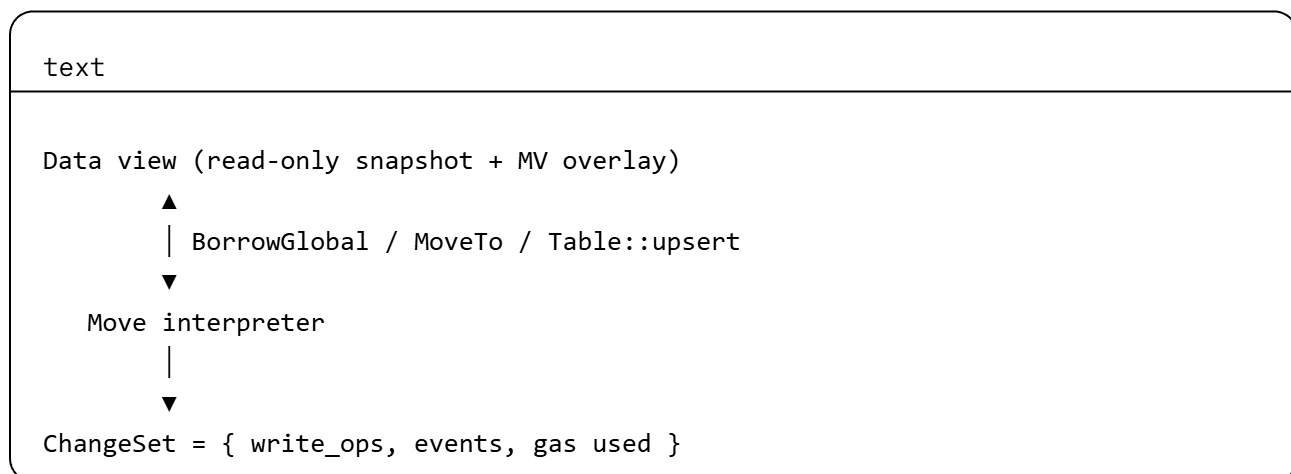
A user account is an address that holds:

- Account resource (sequence number, auth key)
- coin resources, custom resources
- published modules

`signer` is a privileged value only the prologue can mint for the tx's authenticated addresses (including multi-agent). Modules use `signer` to prove “this address authorized the call,” then `move_to` / `borrow_global_mut`.

Aptos **Objects** are an overlay: one address owns a heterogeneous group of resources so an NFT is not “one resource type per account” but a first-class address. They are still looked up through the same global storage API; Block-STM sees them as keys, not as Sui-style declared objects.

How one tx becomes a write set



Each storage op hits the view:

- **Read** → load from MVMemory (highest writer with `idx < me`) or committed storage; record (`key`, `version`) in the read set.
- **Write** → buffer in the incarnation's write set; published to MVMemory when the incarnation finishes.
- **Delete** → write op with deletion flag.

If the view returns “dependency / ESTIMATE,” the adapter turns that into a VM error the scheduler treats as **suspend / retry**, not as a user-level `abort`. User `abort` codes are different: they are part of the deterministic output (failed tx still pays gas, still consumes a sequence number).

Gas is charged per opcode, per byte of IO, and for storage slots created/deleted. The

Gas is charged per opcode, per byte of IO, and for storage slots created/deleted. The epilogue fails the tx if the budget is exceeded. Outputs must be identical on every validator: no clocks, no hashmap iteration order, no floating point.

Transaction payloads the VM understands

Typical Aptos user tx:

text
sender, sequence or orderless nonce payload: EntryFunction Script Multisig Module bundle gas unit price, max gas secondary signers (multi-agent)

A block also injects **system txs** before users: BlockMetadata (timestamp, proposer, round), optional validator payloads (randomness, JWK). Those run in the same VM and write framework resources (Timestamp , validator set). User txs therefore *read* block time as a resource — a classic hot key if everyone touches it.

Ledger versions are **per transaction**, not only per block. After Block-STM commits, each tx has its own state version and write set.

Parallelism: why the VM does not declare access lists

Move programs can `borrow_global` any address they can name. The touched key set is a **runtime fact**, so Aptos chose OCC:

text
ordinary Move → instrumented view → read/write sets → Block-STM validate

Contrast:

	Aptos Move VM	Sui Move VM
Storage API	Global <code>borrow_global</code> / <code>move_to</code>	No global storage; objects passed in
What the tx names	Function + args (+ signers)	Explicit object IDs / ownership
Parallelism	Inferred after/during execute	Known before execute

Hot shared state Same account/resource → STM abort Shared object → consensus path

Same language family; different execution contract. Sui deleted `move_to / borrow_global` so the scheduler can be pessimistic and skip consensus on owned objects. Aptos kept Diem's global store and put intelligence in Block-STM.

Module publish and reentrancy

Publishing is a privileged tx: verifier + loader + write module bytes. Friends and visibility (`public`, `public(friend)`, `entry`, `view`) are enforced at verify/link time.

Call dispatch is **static** (plus Move 2.2 function values with extra reentrancy rules). There is no EVM-style `CALL` into arbitrary bytecode with a shared mutable account. Reentrancy exists only where you pass function values or call back into a module that already holds a `&mut` — the verifier and Aptos rules constrain this. That keeps speculative execution saner than a parallel EVM.

When a tx **publishes modules**, Aptos V2 scheduler runs a separate **cold validation** path: the module cache cannot push-invalidate every other worker, so one worker re-checks module read sets for the suffix of the block. Ordinary resource writes still go through MVMemory.

Failure and determinism

Outcome	State	Gas / seq
Success	Write set applied	Charged, seq++
User abort / out of gas	No user writes (epilogue may still run)	Charged, seq++
Prologue fail	Discarded	Usually not sequenced
Speculative read error	Incarnation discarded; retry	Not a ledger event

Speculative failure is invisible on chain. Only the last successful incarnation's effects (or a user-abort output) become a versioned write set.

Execution pipeline on a validator

text

ordered block

→ Block-STM workers

each: AptosVM.prologue + execute + epilogue

storage = TxnView(MVHashMap, committed state)

→ validate read sets

→ commit prefix write sets to Jellyfish / state DB

→ per-tx versions + events

The VM is **single-threaded per incarnation**. Parallelism is many incarnations, not SIMD inside one swap. Tables and objects just create more keys, which is good for STM if they do not collapse onto one resource.

Implications if you were designing the game chain

- Move VM gives you **linear assets** and a **safe interpreter** for free.
- Aptos-style global storage + Block-STM = no access lists, but shared Marketplace resources will abort.
- Sui-style Move = better fit for “sword is an object,” worse fit for “read any account.”
- Either way the VM must expose a **view** that records every storage key; that is the only integration Block-STM needs.
- Keep natives deterministic; put non-determinism (VRF, matchmaking) outside the interpreter.

The execution model in one sentence: **verified stack bytecode + resource abilities + adapter-mediated global storage**, with Aptos treating storage keys as an implicit access list collected during interpretation so Block-STM can replay the block as if it had been serial